

Universidad de Concepción

Facultad de Ingeniería
Departamento de Ingeniería Informática y Ciencias de la Computación

Estructuras de Datos y Algoritmos Avanzados (2026-2)

Boletín 03: String Matching

Autor:
Bryan Eliseo Aguirre Fuentes

Matrícula:
2024443402

Profesor:
Dr. José Fuentes

Ayudante:
Leonardo Lovera

Concepción, Chile
5 de noviembre de 2026

Índice

1. Resumen **1**
1.1. Propósito y alcance del estudio 1
1.2. Estructuras y algoritmos estudiados 1

2. Análisis teórico **2**
2.1. Búsqueda por fuerza bruta 2
2.2. Knuth–Morris–Pratt (KMP) 2
2.3. Rabin–Karp 2
2.4. Suffix Array 3
2.5. Comparación de complejidades 3

1. Resumen

1.1. Propósito y alcance del estudio

Este estudio compara experimentalmente cuatro métodos de búsqueda de patrones en texto: búsqueda por fuerza bruta, Suffix Array, Knuth–Morris–Pratt (KMP) y Rabin–Karp. La fuerza bruta se incluye como **línea base** para contextualizar el desempeño de los métodos estudiados. Para efectos de los experimentos, *búsqueda* significa encontrar y contar todas las ocurrencias del patrón en el texto, no devolver sus posiciones. Se analiza cómo influyen el largo del texto, el largo del patrón y el número de ocurrencias en el tiempo de búsqueda; además, se consideran el costo de construcción del índice y el consumo de memoria. Los resultados se interpretan junto con las complejidades teóricas para identificar los escenarios en que conviene cada alternativa.

1.2. Estructuras y algoritmos estudiados

Fuerza bruta: Revisa cada posición posible del texto y compara desde allí los caracteres con el patrón. No requiere preprocesamiento y sirve como referencia sencilla, aunque puede repetir comparaciones.

Suffix Array: Ordena las posiciones iniciales de los sufijos del texto. Para buscar un patrón, localiza mediante búsqueda binaria el intervalo de sufijos que comienza con él y cuenta sus ocurrencias. Su índice requiere una etapa de construcción previa.

KMP: Busca el patrón recorriendo el texto y usando información de prefijos del patrón para evitar comparaciones repetidas. Su búsqueda tiene tiempo lineal en el largo del texto y del patrón, sin requerir un índice persistente.

Rabin–Karp: Compara primero valores hash del patrón y de ventanas del texto; cuando coinciden, verifica los caracteres para confirmar la ocurrencia. El hash de las ventanas se actualiza incrementalmente y su desempeño depende, entre otros factores, de las colisiones.

2. Análisis teórico

Sea T el texto y P el patrón. Denotamos sus longitudes por $n = |T|$ y $m = |P|$, respectivamente. Excluimos el patrón vacío ($m = 0$), cuyo número de coincidencias depende de la convención adoptada. Analizamos el caso usual $1 \leq m \leq n$; si $m > n$, las implementaciones devuelven cero inmediatamente. Por *búsqueda* entendemos contar todas las ocurrencias, incluidas las superpuestas. Usamos notación $O(\cdot)$ para expresar cotas superiores y $\Theta(\cdot)$ cuando la cota es ajustada. El espacio se expresa en palabras y no incluye la entrada. Como los métodos devuelven un contador, el espacio de salida es $O(1)$; si devolvieran todas las posiciones, habría que sumar $O(z)$, donde z es el número de ocurrencias.

2.1. Búsqueda por fuerza bruta

El algoritmo prueba cada posición de T y compara desde allí los caracteres de P, avanzando una posición tras cada intento.

- **Preparación:** $\Theta(1)$; no construye estructuras previas.
- **Mejor caso de búsqueda:** $\Theta(n - m + 1)$, cuando cada intento falla al comparar el primer carácter. Una coincidencia inicial puede añadir hasta $O(m)$ comparaciones, pero el algoritmo debe revisar las posiciones restantes para contar todas las ocurrencias.
- **Peor caso de búsqueda:** $\Theta((n - m + 1)m)$, si cada ventana coincide en casi todo el patrón antes de fallar o confirmarse.
- **Memoria auxiliar:** $\Theta(1)$.

2.2. Knuth–Morris–Pratt (KMP)

KMP construye una tabla que indica qué prefijo del patrón puede conservarse después de una discrepancia. Así desplaza el patrón sin retroceder en el texto. La implementación construye la tabla en cada llamada y luego recorre el texto; tras una coincidencia, también usa la tabla para conservar posibles coincidencias superpuestas.

- **Preparación e inicialización:** $\Theta(m)$ tiempo y memoria para construir la tabla de prefijos.
- **Mejor caso de búsqueda:** $\Theta(n)$.
- **Peor caso de búsqueda:** $\Theta(n)$; el recorrido avanza por el texto y los retrocesos del patrón son amortizadamente lineales. Incluida la preparación, el tiempo es $\Theta(n + m)$.
- **Memoria auxiliar:** $\Theta(m)$, por la tabla.

2.3. Rabin–Karp

Rabin–Karp calcula el hash del patrón P y el de la primera ventana de T durante la preparación. La búsqueda verifica esa primera ventana y luego desplaza la ventana, actualizando su hash en tiempo constante. Compara caracteres para confirmar cada coincidencia de hash.

- **Preparación e inicialización:** $\Theta(m)$ tiempo para calcular ambos hashes: el de P y el de la primera ventana de T.

- **Mejor caso de búsqueda:** $\Theta(n - m + 1)$, cuando el hash de P difiere del hash de todas las ventanas examinadas. No hay verificaciones carácter a carácter y cada ventana se procesa en tiempo constante.
- **Peor caso de búsqueda:** $\Theta((n - m + 1)m)$, si muchas ventanas requieren una verificación completa, por colisiones o coincidencias reales superpuestas.
- **Memoria auxiliar:** $\Theta(1)$.

El peor caso puede ocurrir por colisiones de hash o por coincidencias reales en muchas ventanas. Como se usan parámetros de hash fijos y no se supone una distribución aleatoria de las entradas, no se afirma una garantía de tiempo esperado. Se asume que las operaciones con el hash modular cuestan tiempo constante.

2.4. Suffix Array

La estructura ordena los sufijos de T mediante duplicación de prefijos: en cada ronda duplica el largo del prefijo ordenado y actualiza clases con conteo. Para contar un patrón, realiza dos búsquedas binarias que delimitan los sufijos que comienzan con P; no recorre individualmente las z ocurrencias.

- **Preparación (construcción):** $O(n \log n)$ tiempo en el peor caso y $\Theta(n)$ memoria, mediante duplicación de prefijos y ordenamiento por conteo en cada ronda. Esta implementación ejecuta todas las rondas, por lo que su construcción toma $\Theta(n \log n)$.
- **Mejor caso de búsqueda:** $\Theta(\log n)$, si las comparaciones binarias terminan al revisar el primer carácter.
- **Peor caso de búsqueda:** $O(m \log n)$, pues cada comparación puede revisar hasta m caracteres. Es la búsqueda binaria básica, sin optimizaciones basadas en LCP; contar el intervalo cuesta $O(1)$.
- **Memoria:** $O(1)$ auxiliar por consulta; el índice persistente ocupa $\Theta(n)$ y la memoria temporal de construcción, $\Theta(n)$.

La memoria depende de los largos de entrada, no de su contenido; por ello no cambia entre mejor y peor caso.

2.5. Comparación de complejidades

Los tiempos de búsqueda incluyen el conteo. En Rabin–Karp, la preparación incluye el hash de P y el de la primera ventana de T. La preparación del Suffix Array corresponde a su construcción; se distingue el espacio auxiliar por consulta del índice persistente y de la memoria temporal de construcción.

Método	**Preparación**	**Mejor búsqueda**	**Peor búsqueda**	**Memoria auxiliar / índice**
Fuerza bruta	$\Theta(1)$	$\Theta(n - m + 1)$	$\Theta((n - m + 1)m)$	$\Theta(1)$
KMP	$\Theta(m)$	$\Theta(n)$	$\Theta(n)$	$\Theta(m)$
Rabin–Karp	$\Theta(m)$	$\Theta(n - m + 1)$	$\Theta((n - m + 1)m)$	$\Theta(1)$
Suffix Array	$\Theta(n \log n)$	$\Theta(\log n)$	$O(m \log n)$	$O(1)$ consulta; $\Theta(n)$ índice persistente; $\Theta(n)$ temporal al construir